

Cognitive Fusion Mechanism: Integration of LLM and Memory

Witold Warchoř

June 13, 2026

1 Overview

The fusion mechanism projects two independent information streams into a shared 64 dimensional space: the LLM embedding query and episodic memory entries. The design prioritizes information preservation and prevents any single stream from drowning out the other through deterministic, non-learnable feature extraction followed by banded assembly and light context-modulated mixing. Neuron graph reasoning is handled by the Python-side `_NeuronGraphReasoner` (GAT) and enters the fusion head as a separate token sequence rather than through this C-side pipeline.

2 Input Streams

2.1 LLM Query Stream

The LLM provides a dense embedding vector:

$$\mathbf{q}_{\text{raw}} \in \mathbb{R}^{d_{\text{llm}}} \quad (1)$$

where d_{llm} is the model’s embedding dimension (variable). This stream represents the current model query or context.

2.2 Memory Stream

Episodic memory entries consist of a rich vector and an importance weight:

$$\mathbf{m}_i = (\mathbf{v}_i, \alpha_i, \tau_i) \quad \text{where} \quad \mathbf{v}_i \in \mathbb{R}^{2N+d_{\text{input}}} \quad (2)$$

where $N = \text{MAX_NEURONS}$ and $d_{\text{input}} = \text{INPUT_SIZE}$. Up to $M = 1200$ entries may be stored. The vector combines prior neuron state (implicit history) and external input features.

3 Projection Mechanism

3.1 Deterministic Random Projection

Rather than learned weights, projections use a seeded pseudo-random matrix fixed across runs. A projection coefficient is computed as:

$$P(r, c) = \frac{\text{bits}(h(r, c)) \& 0xffffffff}{2^{23}} - 1.0 \quad (3)$$

where $h(r, c)$ applies splitmix64-style mixing:

$$z = (r \ll 32) \oplus (c \cdot 2654435761 + 1) \quad (4)$$

$$z \leftarrow z + 0x9e3779b97f4a7c15 \quad (5)$$

$$z \leftarrow (z \oplus (z \gg 30)) \times 0xbf58476d1ce4e5b9 \quad (6)$$

$$z \leftarrow (z \oplus (z \gg 27)) \times 0x94d049bb133111eb \quad (7)$$

$$z \leftarrow z \oplus (z \gg 31) \quad (8)$$

This ensures:

- Every input dimension feeds every output dimension (dense mixing)
- No information is dropped via blocking or striding
- The matrix is reproducible without storage
- Variance is well-distributed across output dimensions

3.2 Full Projection

An n -dimensional source is projected into the 64 dimensional space:

$$\text{proj}(\mathbf{x})_i = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} x_j \cdot P(i, j) \quad \forall i \in [0, 64) \quad (9)$$

The $\frac{1}{\sqrt{n}}$ scaling follows Johnson-Lindenstrauss and stabilizes variance regardless of source dimension.

3.3 Banded Projection

A source is projected into a contiguous band $[o, o + \ell)$ of the output while leaving other positions untouched:

$$\text{proj_band}(\mathbf{x}, o, \ell, \text{seed})_i = \begin{cases} \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} x_j \cdot P(i + \text{seed}, j) & \text{if } i \in [o, o + \ell) \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

Each band uses a different seed (1009 for the query band, 3001 for the memory band) so bands occupy distinct subspaces and do not share a projection.

4 Processing Pipeline

4.1 Stage 1: LLM Query with Text Integration

The LLM embedding is projected and normalized:

$$\mathbf{q} = \text{LayerNorm}(\text{proj}(\mathbf{q}_{\text{raw}})) \quad (11)$$

If an input sentence embedding is provided, it is projected separately, normalized, and blended at 0.5 strength:

$$\mathbf{q} \leftarrow \text{LayerNorm}(\mathbf{q} + 0.5 \cdot \text{LayerNorm}(\text{proj}(\mathbf{t}_{\text{raw}}))) \quad (12)$$

This allows the input sentence to directly influence the query band without being gated away by memory blending.

4.2 Stage 2: Top-K Memory Attention

Memory entries are scored by dot-product similarity to the query, and only the top- K entries ($K = \min(M, 32)$) are used. This prevents softmax collapse to uniform weights when entry importances are similar.

4.2.1 Blending Prior and Memory

Each entry is weighted-blended with a default system prior:

$$\mathbf{w}_i = (1 - \beta)\mathbf{d} + \beta\mathbf{v}_i \quad \text{scaled by} \quad \tilde{\mathbf{w}}_i = \alpha_i \cdot \mathbf{w}_i \quad (13)$$

where $\beta = \text{mem_weight_ratio} \in [0, 1]$ controls the blend ($\beta = 0$ uses pure default, $\beta = 1$ uses pure stored memory).

4.2.2 Key Projection and Scoring

Each blended vector is projected and scored:

$$\mathbf{k}_i = \text{LayerNorm}(\text{proj}(\tilde{\mathbf{w}}_i)) \quad , \quad s_i = \frac{1}{\sqrt{64}} \langle \mathbf{q}, \mathbf{k}_i \rangle \quad (14)$$

4.2.3 Top-K Selection

The top $K = \min(M, 32)$ entries are selected by score. Their scores are normalized via softmax:

$$\pi_i = \text{softmax}(s_i) = \frac{\exp(s_i - \max_j s_j)}{\sum_j \exp(s_j - \max_j s_j)} \quad (15)$$

4.2.4 Memory Vector Aggregation

The memory contribution is the weighted sum:

$$\mathbf{m} = \text{LayerNorm} \left(\sum_{i \in \text{top-K}} \pi_i \mathbf{k}_i \right) \quad (16)$$

4.3 Stage 3: Banded Assembly

Each stream is re-projected into its own contiguous band:

$$\text{fused}[0 : 32] = \text{proj_band}(\mathbf{q}, 0, 32, 1009) \quad (17)$$

$$\text{fused}[32 : 64] = \text{proj_band}(\mathbf{m}, 32, 32, 3001) \quad (18)$$

where $\text{BAND_Q} = 32$, $\text{BAND_M} = 32$, and $\text{FUSION_DIM} = \text{BAND_Q} + \text{BAND_M} = 64$.

Key property: Each stream owns a disjoint set of dimensions. No gating mechanism controls how much of each stream appears; instead, the downstream fusion transformer learns to weight the bands as needed.

4.4 Stage 4: Context-Modulated Cross-Band Mixing

A light mixing term is applied based on context:

$$\gamma = \text{sigmoid}(2 \cdot \text{context_factor} - 1) \times 0.5 \quad (19)$$

For each dimension, a partner dimension is mixed in:

$$\text{cross}[i] = \text{fused}[i] + \gamma \cdot \text{fused}[(i + 32) \bmod 64] \quad (20)$$

This introduces low-amplitude interactions between bands without allowing dominance of any stream.

4.5 Stage 5: Output Normalization

The final vector is layer-normalized:

$$\mathbf{out} = \text{LayerNorm}(\mathbf{cross}) \quad (21)$$

5 Layer Normalization

Layer normalization standardizes a vector to zero mean and unit variance:

$$\mu = \frac{1}{n} \sum_{i=0}^{n-1} v_i \quad (22)$$

$$\sigma^2 = \frac{1}{n} \sum_{i=0}^{n-1} (v_i - \mu)^2 \quad (23)$$

$$\text{LayerNorm}(\mathbf{v})_i = \frac{v_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (24)$$

where $\epsilon = 10^{-6}$ prevents division by zero.

6 Design Rationale

6.1 Why Deterministic Projection?

The C implementation is a deterministic feature extractor, not a learner. Since gradients are killed at the ctypes boundary, learnable mixing must occur in the downstream Python fusion transformer. The C side’s role is to produce a rich, stable, non-degenerate feature vector.

6.2 Why Banded Architecture?

By carving the output into two disjoint bands, each stream is guaranteed representation. The transformer downstream learns to weight each band appropriately. Neuron graph information enters the fusion head through the Python-side GAT token sequence, which preserves the full graph topology rather than collapsing it into a fixed-size projection.

6.3 Why Top-K Memory Attention?

Attending over all M memory entries with similar importances causes softmax to collapse to uniform weights, reducing the memory vector to noise averaging. Top-K selection ($K = 32$) ensures the attention distribution is peaked and informative even at high memory capacities ($M = 1200$).

6.4 Why Project Bands Separately?

Each band uses a different seed in the pseudo-random projection. This ensures the two streams occupy distinct subspaces and their projected features do not interfere.

7 Output Characteristics

- **Dimension:** 64 (= BAND_Q + BAND_M = 32 + 32)
- **Bands:** Query (dims 0–31), Memory (dims 32–63)
- **Normalization:** Layer-normalized, mean 0, variance 1
- **Mixing:** Light cross-band mixing at ≤ 0.5 strength, controlled by context
- **Information:** Both C-side streams present in balanced proportion; neuron information enters the fusion head separately via GAT tokens